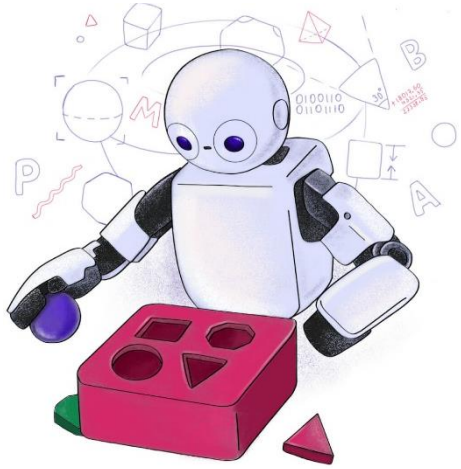
TP558 - Tópicos avançados em Machine Learning

Genetic Algorithms

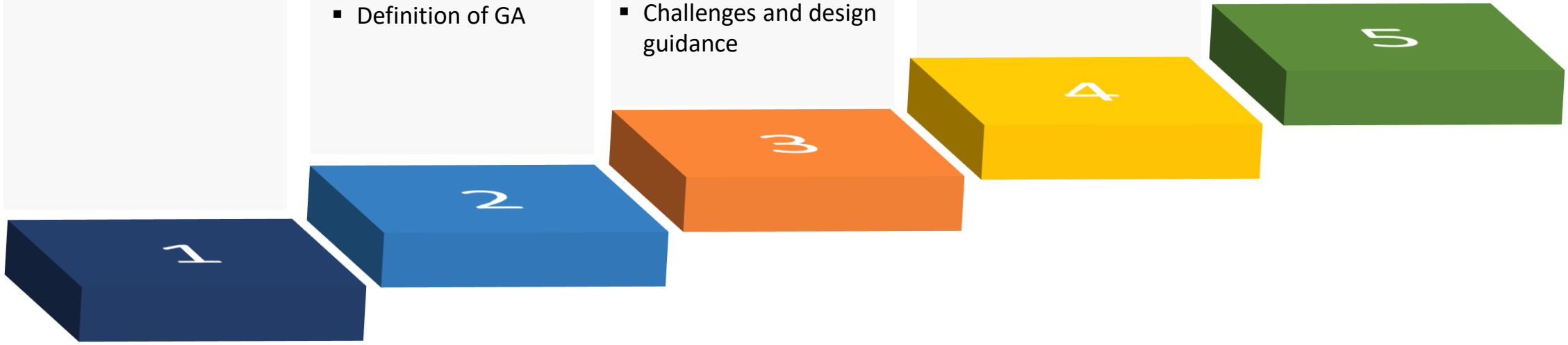


Inatel

Hanieh Memarian
hanieh.memarian@dtel.inatel.br

Outline

Introduction	Definition & Development of GA	Challenges	Article	Future work & Conclusion
▪ Background ▪ Basic problem	▪ What is GA ▪ Why is GA Useful ▪ Definition of GA	▪ Strengths of GA ▪ Limitations of GA ▪ Challenges and design guidance	▪ Algorithm ▪ Simulation Results	▪ Conclusion ▪ Future Directions



Introduction

Background of GA

Biological inspiration

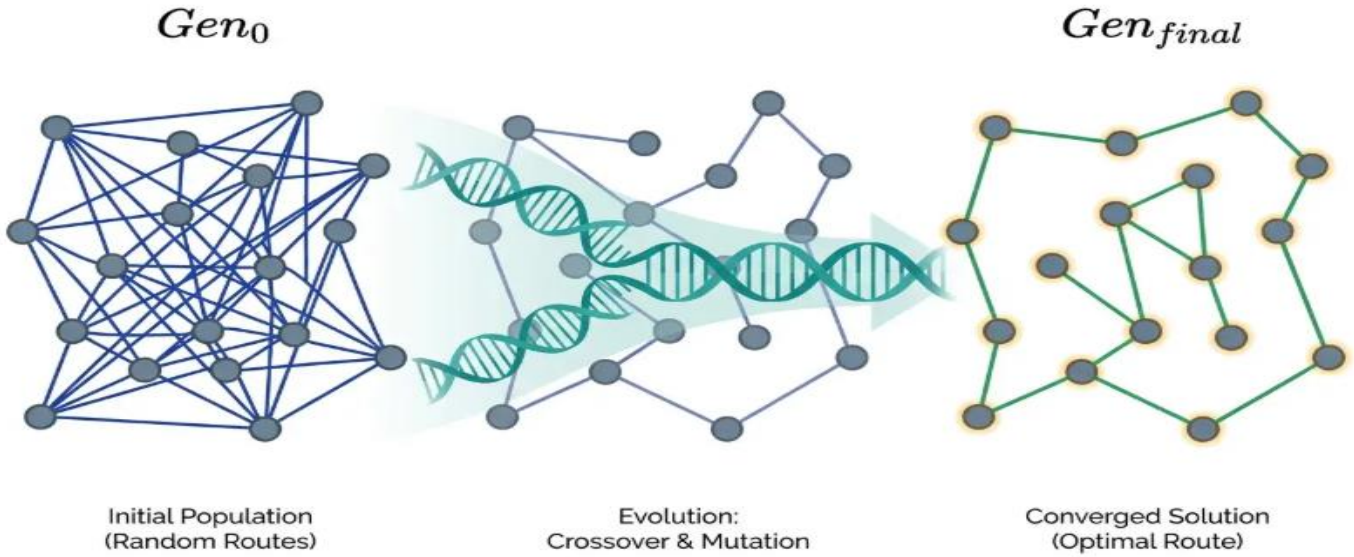
A metaheuristic that imitates evolution: chromosomes, mutation, crossover and natural selection.

Origins

Turing proposed a learning machine in 1950; Holland developed GAs from the 1960s and published in 1975.

Classical GA

A population of binary strings; a string's fitness decides its influence on future search.



Basic problem search and optimization

Huge search spaces

A string of L bits spans 2^L points, so enumeration quickly becomes impossible.

No derivatives needed

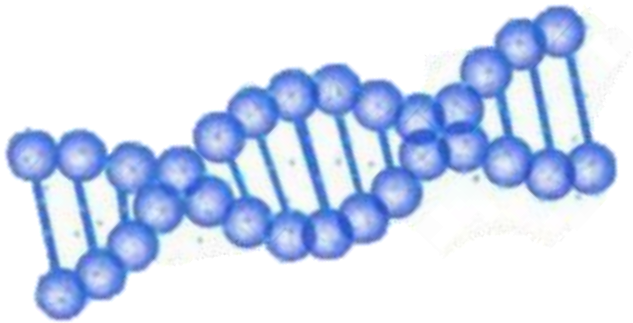
The objective can be a simulator, noisy or discontinuous; only evaluations are required.

Moving targets

In dynamic settings such as wireless, the optimum keeps shifting: “dancing landscapes”.

Constrained design

Network design mixes placement, routing and capacity limits; exact solvers slow down on large instances.



Definition and Development of GA

What is a GA?

1 Initialize

Random or seeded population of chromosomes

2 Evaluate

Compute fitness of every chromosome

3 Select

Fitter chromosomes are likelier parents

4 Recombine

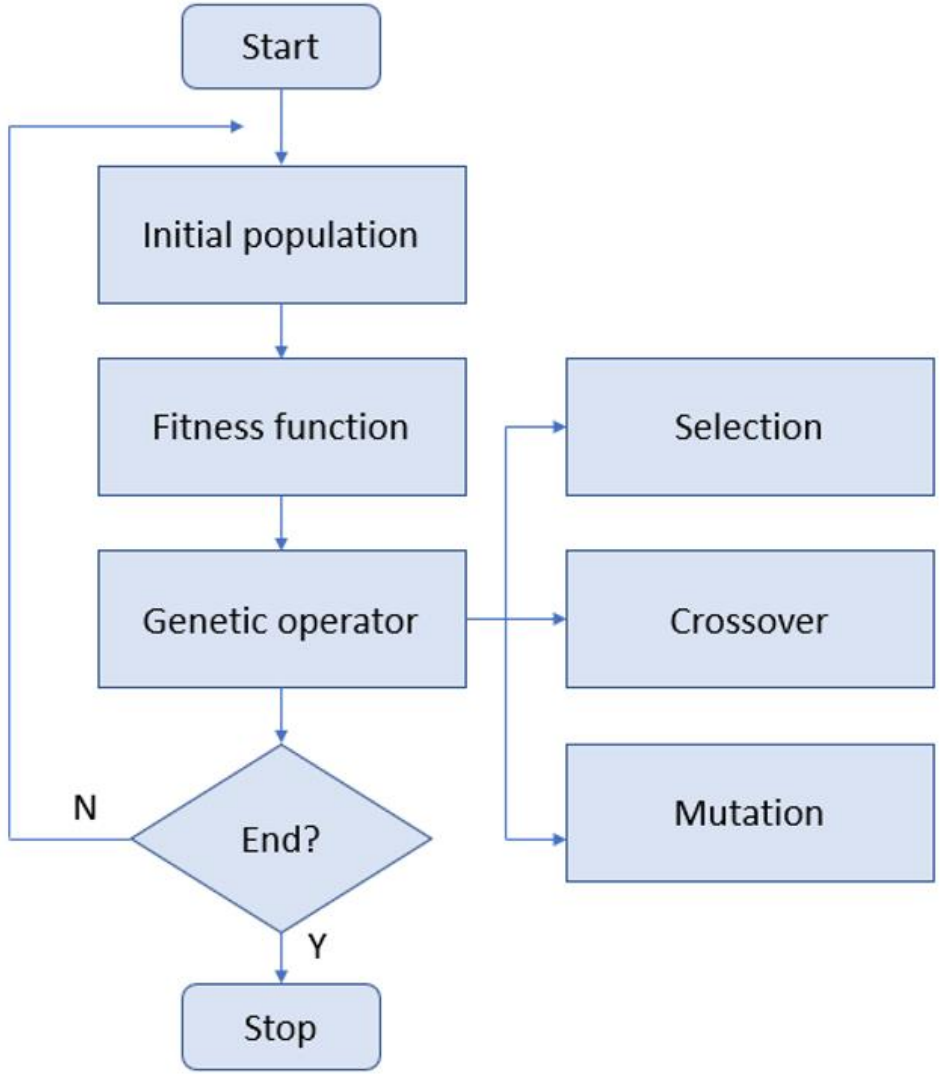
Crossover swaps parent segments

5 Mutate

Rare random changes keep diversity

6 Repeat

Repeat until a termination criterion is met; elitism can keep the best individuals unchanged.



Why is a GA useful?

Rugged landscapes

Highly multimodal, discrete or discontinuous functions.

Interacting parameters

Nonlinear dependencies (epistasis) that defeat simpler methods.

Hard combinatorial problems

Good approximations for NP-complete problems.

Proven real-world use

Antenna, VLSI and scheduling design; telecom network design with 31% and 52.2% cost cuts (A4).

Do not use a GA if a hill-climber works well.

schemata and building blocks

Schemata

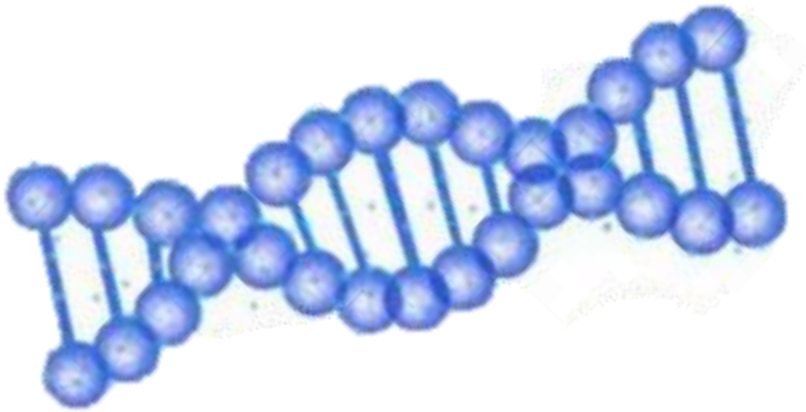
A string of length L is an instance of 2^{L-1}schemata.

Schema theorem

Short, low-order, above-average schemata receive exponentially more samples.

Building blocks

Crossover recombines good short pieces into better strings; implicit parallelism.



Definition of GA

$$P(H,t+1) \geq P(H,t)\,\frac{f(H,t)}{\bar{f}}\,[1 - P_c\,\frac{\Delta(H)}{L-1} - O(H)\,P_m]$$

H: schema; **f(H,t)**: mean fitness of its instances; $\bar{f}$: population mean; **Δ(H)**: defining length; **O(H)**: order; P_c, P_m : crossover and mutation probabilities; **L**: string length.

A GA is a stochastic, population-based search that applies selection, crossover and mutation to evolve encoded candidate solutions toward higher fitness.

Challenges

Strengths of GA

Generality

Works on a wide range of problems and when the objective is only evaluated.

Adaptive, online

Mutation explores while selection exploits; suits changing conditions.

Beats exact solvers at scale

On large PON instances a GA outran ILP in run time (A4).

Parallel, scalable

Population structure maps to island and master-slave models.

Multi-objective

Pareto-front search with NSGA-II and variants.

Easy to implement

Swapping and shifting genes; fits DSPs and FPGAs.

Limitations of GA

No guarantee

No proof of global optimality, even when a GA beats ILP on run time (A4).

Premature convergence

Genetic drift with small or uniform populations.

Costly evaluation

Thousands of fitness calls; evaluation time is the usual bottleneck.

Not real-time safe

Stochastic: run time varies and is unbounded.

Deception, epistasis

Hitchhiking and deceptive functions can mislead search.

No Free Lunch

No method is best across all problems.

Challenges and design guidance

Premature convergence	Fitness sharing, crowding, mutation; scaled or rank selection
Hamming cliffs (binary integers)	Use Gray codes so neighbours differ by one bit
Parameter tuning	Pop. 50-100; P_c 0.6, P_m 0.001 (pop. ~100); P_c 0.9, P_m 0.01 (pop. ~30)
Slow fitness evaluation	Fitness approximation, caching, parallel and island GAs
Settings in practice	[4]: pop. 100, 100 generations, tournament 5, 10% elitism; cap generations near 2.5 × pop. ([1])

Article

Genetic Algorithms in Wireless Networking

Why use GA in wireless networks?

Dynamic channels

Optima keep changing; GAs adapt online and satisfice rather than optimize.

Conflicting objectives

BER, power, SINR and throughput are optimized jointly by multi-objective GAs.

Black-box fitness

No explicit objective formula is needed, only an evaluation.

Application areas surveyed

Routing	Ahn & Ramakrishna [23]; Yang et al. [34]
QoS	Xiao et al. [73] (admission control); Roy et al. [49]
Load balancing	Scully & Brown [70] (WLAN)
Bandwidth allocation	Lin & Labeau [75]
Channel allocation	Ngo & Li [98]; Friend et al. [17]
Localization	Zhang et al. [99]; Tam et al. [101]
Network planning, placement	Pries et al. [120]; Ghosh et al. [122]; Xhafa et al. [105]

GA by network type

Sensor networks	Coverage, clustering, sleep scheduling (Jia; Ferentinos)
Mesh networks	Router placement for coverage and connectivity (Xhafa et al.)
Cognitive radio	Cognitive engine with WSGA; island GA for channels
Cellular networks	Relay stations, base-station sleep modes (Chiaraviglio et al.)
Software-defined	Small-cell backhaul resource manager (Bojic et al.)

Hybrid GAs

Game theory
Evolutionary games for sensing, power control and routing.

Reinforcement learning
GA as policy-space search over encoded policies.

Fuzzy logic
GA tunes membership functions and rules.

Quantum GA
Qubit chromosomes; more effective on larger inputs.

Memetic
Local search on each member: stronger but costlier; used in A4.

Other metaheuristics
ACO routing, GA-trained neural network weights.

Main problem
automatic network design

Rising demand

Streaming, cloud and gaming push providers to fibre (FTTH, FTTB).

High investment

Ducts and cables dominate cost; good design can save about 30%.

Slow manual design

Splitters, headend, PDOs, routes, distances and optical budget: days of effort.

Design a GPON / fiber-to-the-home network at equipment level, on an existing physical infrastructure, at minimum cost.

Problem formulation GPON/FTTH design

One OLT feeder fibre feeds a 1:N splitter that serves ONUs (client modems); splitting is centralised or cascaded.

Decisions

Where to place capacitated PDOs (street cabinets), which client connects to which PDO, splitter ratio and level.

Constraints

12 ports per PDO with 10% spare; drop cable up to 85 m; reach 20 km and optical budget (about 0.35 dB/km); drop cables must not cross distribution cables.

Objective

Minimize total cost of PDOs, drop cable and distribution cable, avoid buried routes, serve every client's demand.

Algorithm
two-level GA with local search

Two-level representation

Level 1: binary list, 1 = PDO placed on that equipment. Level 2: integer list, client → PDO id, −1 if unassigned.

Variation operators

Bit-flip mutation and uniform crossover act on level 1; level 2 is then repaired for affected homes.

Local search

Clients first join their closest PDO; hill-climbing swaps of drop links fix the rest, within range and port limits.

Selection

Tournament selection (size 5) with 10% elitism, 100 generations on a population of 100.

Fitness function and experimental setup

$$C_{Mat} = C_{Drop} \cdot N_{Drop} + C_{Dist} \cdot N_{Dist} + C_{PDO} \cdot N_{PDO}$$
$$f = C_{Mat} + H_{Missing} \cdot P$$

$H_{Missing}$: homes left unconnected; P: penalty set to the cost of one PDO. Drop cable is ×10 costlier for buildings; Dijkstra paths are cached.

Costs and limits

PDO $300; drop cable $2/m; distribution cable $5/m. 12 ports, 10% spare, drop ≤ 85 m, reach 20 km.

GA settings

Pop. 100; 100 generations; bit-flip 2/len; uniform crossover rate 0.85; tournament 5; 10% elitism.

Simulation results

Metric	Map 1 handmade	Map 1 GA best	Map 2 handmade	Map 2 GA best
Number of PDOs	62	17	27	14
Drop cable (km)	2.75	4.18	3.53	3.09
Distribution cable (km)	2.08	1.74	2.76	1.65
Fitness (cost score)	34,505	23,908	50,216	24,828

Cost cut: 31% on Map 1; 52.2% reported for Map 2 (its Table 7 gives about 50.6%). Run time about 30.5 s and 37 s, against hours to days by hand.

Fitness falls fastest in the first 25 generations. Map 1: 103 houses, 5 buildings; Map 2: 76 houses, 11 buildings.

Future work and Conclusion

Conclusion

A GA designed GPON networks 31% and 52.2% cheaper than handmade ones in seconds, with no optimality guarantee.

Future Directions

Distributed multi-objective GAs, wireless-specific knowledge, stronger theory, big-data scaling.

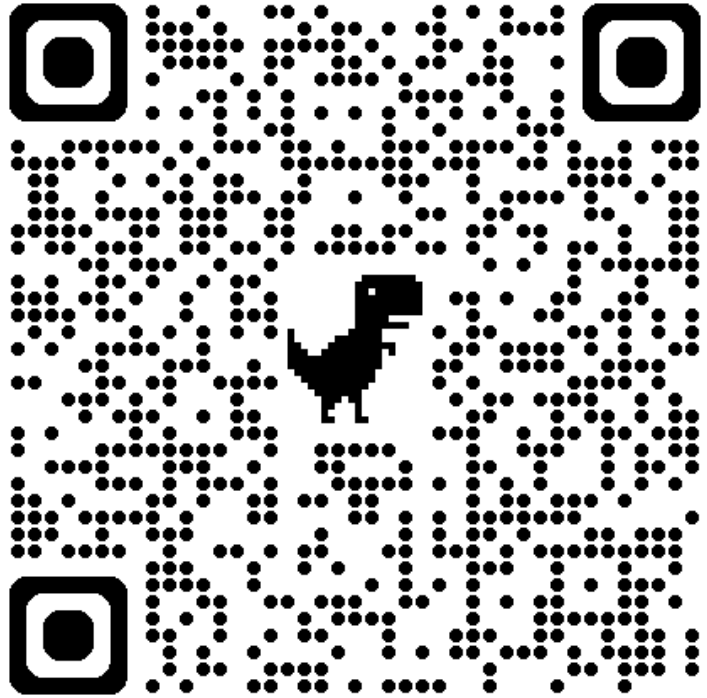
[A1] A1 Mehboob, Qadir, Ali, Vasilakos, “Genetic Algorithms in Wireless Networking,” arXiv:1411.5323, 2014.

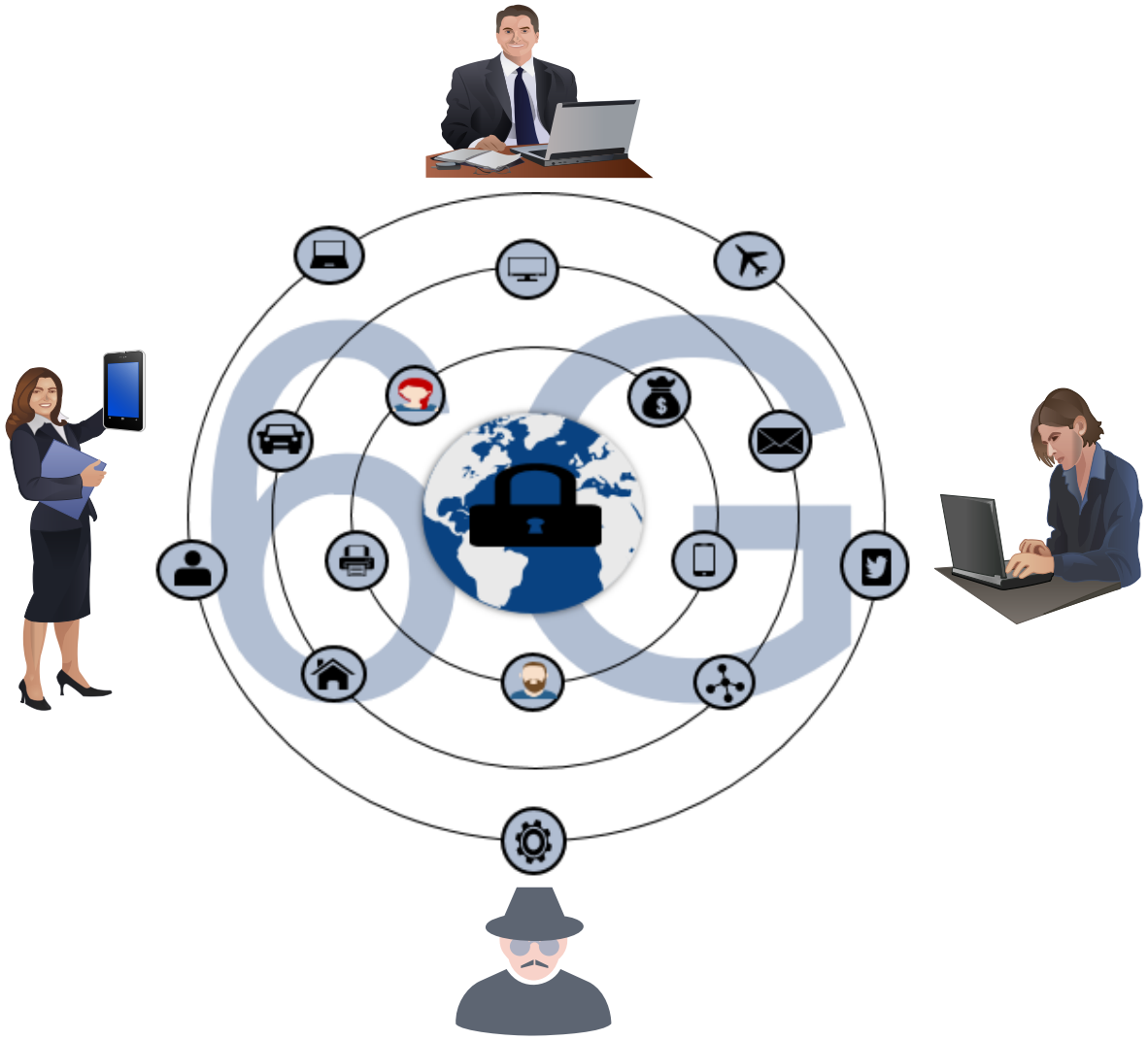
[A2] E. D. Goodman, “Introduction to Genetic Algorithms,” GECCO 2007 tutorial.

[A3] D. Whitley, “A Genetic Algorithm Tutorial,” Statistics and Computing, 1994.

[A4] Correia et al., “Automatic Design of Telecom Networks with Genetic Algorithms,” arXiv:2304.00637, 2023.

https://forms.gle/dT7m8f9vYrYxZ4vy9





Thank you for your attention